Расширенный курс языка C++

К.А. Кутасин, Л. М. Львова

Домашнее задание 1

Требования к программам

1. Для решения каждой задачи предлагается реализовать класс на языке программирования C++.
2. Класс должен называться в точности как указано в задании.
3. Класс должен реализовывать все методы указанные в задании.
4. Исходный код класса должен содержаться в двух файлах: **classname.h**, **classname.cpp**, где **classname** – указанное в задании имя класса.
5. Исходный код должен быть оформлен согласно набору правил и рекомендаций стандарта кодирования GNU.
6. Использовать реализации аналогичных классов из стандартной библиотеки языка в качестве подлежащей реализации запрещено.
7. Проверка задания осуществляется путем использования реализованного класса в тестовой программе. Тестовая программа использует все указанные в задании методы и никакие другие.
8. Если тестовая программа компилируется и корректно работает с приведенной реализацией класса, а реализация соответствует приведенным требованиям, то задание считается выполненным.
9. Если тестовая программа не компилируется или некорректно работает с приведенной реализацией класса, то задание считается не выполненным.
10. Если реализация класса не соответствует приведенным требованиям, то задание считается не выполненным.

Задачи

1. Написать класс **rational**, реализующий представление рационального числа в виде числителя и знаменателя. Класс должен поддерживать два вида инициализации: по числителю и знаменателю и по числу с плавающей точкой. Класс должен поддерживать арифметические операции. Объявление класса приведено ниже:

```
class rational
{
public:
  rational ();
  rational (float value);
  rational (int64_t num, int64_t denom);

  float to_float   () const;
  void  from_float (float val);

  rational &add (const rational &rhs);
  rational &subtract (const rational &rhs);
  rational &multiply (const rational &rhs);
  rational &divide (const rational &rhs);

  // Выдать содержимое класса в соотвествующие ссылки
  void  to_num_denom   (int64_t &num, int64_t &denom) const;
  // Выставить содержимое класса по переданым значениям
```

```
  void  from_num_denom (int64_t num, int64_t denom);

private:
  int64_t numerator{};
  int64_t denominator{};
};
```

2. Написать класс **date_type**, реализующий работу с некоторой датой. Класс должен хранить число секунд с начала эпохи UNIX в качестве текущей даты. Класс должен поддерживать два вида инициализации: по числу секунд с начала эпохи UNIX и по текущей дате в формате число/месяц/год. Класс должен поддерживать получение текущей даты, выставление и получение текущего времени, а так же печать даты и времени в stderr. Объявление класса приведено ниже:

```
class date_type
{
public:
  date_type () = default;
  date_type (int64_t unix_epoch_sec);
  date_type (int dd, int mm, int yy) { set_date (dd, mm, yy); }
  date_type (int d, int m, int y, int hh, int mm, ss)
  {
    set_date (d , m , y );
    set_time (hh, mm, ss);
  }
  void set_date (int dd, int mm, int yy);
  void get_date (int &dd, int &mm, int &yy) const;

  void set_time (int hh, int mm, int ss);
  void get_time (int &hh, int &mm, int &ss) const;

  int64_t get_walltime_sec () const { return m_unix_epoch_sec; }

  void print () const
  {
    int d, m, y = 0;
    get_date (d, m, y);
    int hh, mm, ss = 0;
    get_time (hh, mm, ss);
    fprintf (stderr, "%.2d.%.2d.%.4d %.2d:%.2d:%.2d\n", d, m, y, hh, mm, ss);
  }

private:
  int64_t m_unix_epoch_sec = 0;
};
```

3. Написать класс **block**, реализующий работу с некоторым массивом чисел с плавающей точкой. Класс должен поддерживать два вида инициализации: с выделением памяти внутри класса (размера **size**) и с получением указателя на массив размера **size** (выделенный внешним образом с помощью оператора **new[]**). Класс должен освобождать память в деструкторе **только если он её выделял, внешний указатель освобождать не надо**. Класс должен поддерживать несколько методов взаимодействия с другими экземплярами класса: перестановка, копирование, сумма и разность. Подробное описание методов и объявление класса приведено ниже:

```
class block
{
public:
  block () = default;
  block (size_t size) { allocate_block (size); }
  block (double *start, size_t size) { link_block (start, size); }
  ~block () { free_block (); }
```

```
  // Выделение памяти для класса (возвращает указатель на полученный массив)
  // Выделенный массив надо заполнить нулями
  double *allocate_block (size_t size);

  // Сохранение внешнего указателя в класс
  void link_block (double *start, size_t size);

  // Очистка памяти (только если указатель не внешний)
  void free_block ();

  // Обмен указателями на массив между классами
  void swap_block (block &rhs);

  // Копирование содержимого массива из текущего класса в класс dest
  void copy_to (block &dest);

  // Копирование содержимого массива из класса src в текущий класс
  void copy_from (block &src);

  // Прибавляем не более чем m_size элементов из rhs в хранящийся массив
  // Возвращаем максимальный элемент из полученного массива
  double add_block (const block &rhs);

  // Вычитаем не более чем m_size элементов из rhs из хранящегося массива
  // Возвращаем минимальный элемент из полученного массива
  double sub_block (const block &rhs);

  // Печать не более чем 20 элементов массива в stderr
  // Формат печати каждого элемента:
  // fprintf (stderr, "%e ", m_start[i])
  void print_block ();
private:
  double *m_start{};
  size_t m_size{};
};
```